

CAN201 Report

Team

Yaokai Yang (2359830)
Xiang Lin (2362086)
Xinyang Zhang (2360067)
Yunxuan Wu (2361506)
Haoran Zheng (2359982)

Transparent Traffic Redirection via Header Rewriting in Software-Defined Networks: A Ryu-Based Approach

Yaokai Yang

*School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
yaokai.yang23@student.xjtlu.edu.cn*

Xiang Lin

*School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
xiang.lin23@student.xjtlu.edu.cn*

Xinyang Zhang

*School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
xinyang.zhang23@student.xjtlu.edu.cn*

Yunxuan Wu

*School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
yunxuan.wu23@student.xjtlu.edu.cn*

Haoran Zheng

*School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
haoran.zheng23@student.xjtlu.edu.cn*

Abstract—This paper presents the design and implementation of a transparent traffic redirection mechanism using the Ryu controller within a Mininet environment. We compare a baseline forwarding algorithm against a redirection application that transparently diverts TCP traffic to an alternative host via bidirectional header rewriting. The system ensures client-side transparency by modifying IP and MAC addresses on both forward and reverse paths. Experimental results confirm functional correctness with minimal performance overhead: the average TCP three-way handshake latency is 4.036 ms for redirection compared to 3.328 ms for standard forwarding. These findings validate the feasibility of SDN-based header rewriting for load balancing and secure traffic steering.

I. INTRODUCTION

Software-Defined Networking (SDN) has emerged as a revolutionary paradigm in modern networking, characterized by the decoupling of the control plane from the data plane to enable flexible, programmable, and centralized network management.

This project focuses on leveraging SDN technology to implement traffic forwarding and redirection functionalities within a simplified network topology. The work is structured around the following specific tasks:

- Construct the required SDN network topology using Mininet.
- Design two SDN controller algorithms based on the Ryu framework: the first algorithm enables the forwarding of TCP traffic to Server 1; the second algorithm is used to redirect TCP traffic to Server 2.
- Run Wireshark/Tcpdump on the client host to capture data packets for calculating network latency.

Several challenges are encountered during the implementation process. Firstly, it is necessary to accurately configure

Mininet components to ensure seamless connectivity between nodes. Additionally, in-depth understanding of the SDN packet processing mechanism and idle timeout configuration is required for developing controller applications.

The practical application value of this project is reflected in multiple scenarios: in cloud computing environments, the traffic redirection function can be used for load balancing; in the field of network security, it supports secure traffic steering; furthermore, this solution can also be applied to service migration scenarios to ensure the continuity of client access.

The core contributions of this project include constructing the Mininet topology that meets the requirements, developing two Ryu controllers (for forwarding / redirection) with timeout configuration, and conducting latency comparison analysis through packet capture.

II. RELATED WORK

Unlike traditional networks, SDN decouples the control plane from the data forwarding plane, and centralizes intelligence in a software-based controller that maintains a global view of the network state via the OpenFlow protocol [1], [2]. This programmable interface allows the controller to manipulate the forwarding tables (flow tables) of switches directly. In this project, OpenFlow is essential for installing specific flow entries that modify packet handling behavior, such as rewriting destination MAC addresses to achieve transparent redirection.

To simulate the network environment efficiently, Mininet is employed as the network emulator [3]. Mininet uses Linux process-based virtualization to create a realistic network of hosts, switches, and controllers. It allows for the creation of

custom topologies using Python scripts, providing an environment where real code—including the network topology script (*networkTopo.py*) and socket programs—can run exactly as they would on hardware.

Ryu is a component-based SDN controller framework widely used for developing network management applications [4]. Written in Python, it provides a convenient API for handling OpenFlow messages. In this coursework, the Ryu framework is utilized to implement the control logic scripts (*ryu_forward.py* and *ryu_redirect.py*), enabling the controller to intercept TCP SYN packets and install flow rules to manage traffic flow between the client and servers.

Beyond the enabling technologies, this project is closely related to SDN-based traffic engineering applications, particularly dynamic load balancing and transparent redirection [5]. SDN democratizes traditional networks by allowing the controller to modify packet headers on standard switches. This mechanism allows for the implementation of transparent proxies or server load balancing, where client requests are dynamically distributed to different servers without the client's awareness [6]. This project implements a simplified version of this logic by intercepting TCP SYN packets to redirect traffic flow.

III. DESIGN

This section presents the design of the SDN system developed to support both forwarding and TCP redirection. The discussion is organised around three core components: the architecture of the network, the controller workflow that governs packet handling, and the algorithms that implement redirection at the control plane. The overall design goal is to ensure that every flow's initial packet is visible to the controller, allowing precise flow rule construction and selective manipulation of TCP traffic.

A. System Architecture

The network adopts a minimal SDN topology in which a single OpenFlow switch (*s1*) forms the sole forwarding element between three hosts: the client, Server 1, and Server 2. A Ryu controller communicates with the switch over OpenFlow 1.3 and acts as the central decision entity. All hosts are assigned static IP and MAC addresses in the Mininet topology, ensuring that flow-matching decisions are deterministic. The architecture is shown in Fig. 1; although compact, it provides complete visibility over all traffic.

All packets traverse *s1*; thus, the controller is designed to observe the first packet belonging to each flow. This structural property is essential for implementing selective TCP redirection.

B. Overall System Workflow

The SDN controller operates in a reactive manner and supports two tasks: normal TCP forwarding and transparent TCP redirection. Both tasks share the same overall workflow.

When the switch connects to the controller, a table-miss flow entry is installed to forward all unmatched packets to the

controller. This ensures that the controller receives the first packet of each new flow.

Upon receiving a `PacketIn` message, the controller parses the packet headers and classifies the traffic type. ARP and ICMP packets are handled using normal forwarding logic. For TCP packets, the controller extracts the five-tuple (IP_{src} , IP_{dst} , $port_{src}$, $port_{dst}$, $protocol$) to identify the flow and then determines whether the flow should be normally forwarded or redirected.

Based on this decision, the controller installs the corresponding flow entries and forwards the triggering packet using a `PacketOut` message. After the flow entries are installed, all subsequent packets of the same flow are handled directly by the switch.

C. Normal Forwarding Workflow

This subsection describes the normal TCP forwarding behaviour implemented in *ryu_forward.py*. For TCP flows that are not subject to redirection, the controller relies on MAC learning and per-flow rules to ensure that only the first few packets of each connection are handled in the control plane.

When the switch connects to the controller, a default table-miss entry is installed so that any packet without a matching flow entry is sent to the controller. When the client sends the first TCP SYN segment to Server 1, this SYN triggers a `PacketIn` event. The controller unpacks the Ethernet/IPv4/TCP headers, recognises the packet as IPv4/TCP, and uses the MAC-learning table (populated by earlier ARP/ICMP exchanges) to determine the output port towards Server 1.

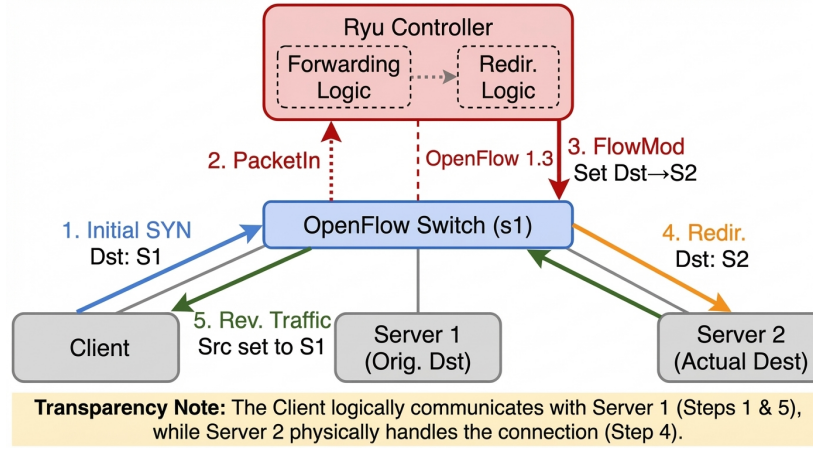
Based on the extracted TCP five-tuple (source/destination IP addresses and TCP ports), the controller installs a new flow-table entry on the switch. This entry matches the TCP flow and uses a 5-second idle timeout so that it expires automatically when inactive. At the same time, the SYN segment is sent back to the switch via a `PacketOut` message and forwarded immediately to Server 1.

When the corresponding SYN/ACK segment from Server 1 first arrives, it again triggers a `PacketIn` event. The controller installs a reverse-direction TCP flow entry, also with a 5-second idle timeout, so that packets from Server 1 back to the client are handled directly by the switch. Once this pair of unidirectional rules is installed, the remaining TCP three-way handshake and subsequent application data are matched entirely in the data plane, without further controller intervention.

D. TCP Redirection Workflow

This subsection describes the transparent TCP redirection behaviour implemented in *ryu_redirect.py*. In this scenario, the controller preserves normal ARP/ICMP connectivity via MAC learning and simple forwarding rules, but modifies how specific TCP flows destined for Server 1 are handled so that they are transparently redirected to Server 2.

When the switch connects, a default table-miss entry is installed so that any packet without a matching flow entry



Server 2. Because the switch applies the rewrite rules after the first packet, no additional controller load is introduced once the flow is established.

IV. IMPLEMENTATION

A. Development Environment & Tools

The system was developed and tested in a VirtualBox virtual machine running Ubuntu (64-bit). The VM was hosted on a Windows desktop equipped with an Intel Core i5-14600KF CPU and 32 GB RAM. The data plane is emulated using Mininet with Open vSwitch, while the control plane is implemented using the Ryu framework (Python 3.8). PyCharm was used as the IDE to develop the modular Python scripts, utilizing Ryu’s APIs for parsing Ethernet/IPv4/TCP headers and managing OpenFlow 1.3 messages.

B. System Logic and Workflow

The comprehensive implementation workflow, covering topology initialization, connectivity verification, and the distinct logic for forwarding and redirection, is illustrated in Fig. 2. The `networkTopo.py` script initializes the topology, configuring the subnet (10.0.1.0/24) and setting the switch to secure fail mode. A table-miss flow entry is installed immediately upon controller connection to ensure the controller inspects the first packet of every new flow.

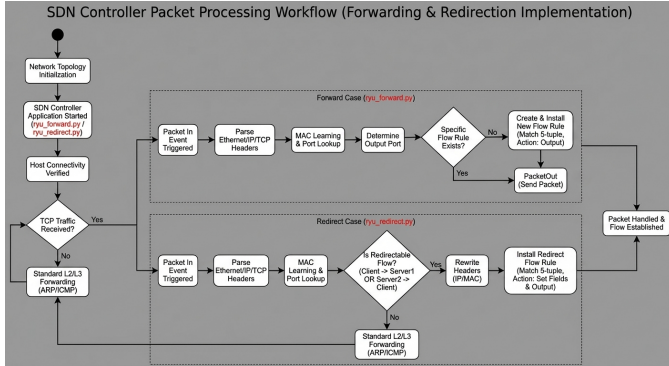


Fig. 2: The detailed program flowchart of the SDN implementation, illustrating the decision logic for both Forwarding and Redirection cases.

As shown in the flowchart, the system branches based on the loaded controller application:

- **Forwarding Case (`ryu_forward.py`):** The controller detects the initial TCP SYN packet destined for Server 1. It performs a MAC–port lookup and installs per-direction flow entries (forming a bidirectional flow) with a 5-second idle timeout. Subsequent packets are handled entirely by the switch.
- **Redirection Case (`ryu_redirect.py`):** When a TCP SYN packet destined for Server 1 (10.0.1.2) is intercepted, the controller executes precise header rewriting. Using the `OFPActionSetField` class, the controller constructs a flow-mod message that rewrites the destination IPv4 to Server 2 (10.0.1.3) and the destination

MAC to 00:00:00:00:00:02 before forwarding to the corresponding port. Crucially, for the reverse path (Server 2 → Client), a symmetric per-direction rule is installed to rewrite the source IPv4 and MAC addresses back to Server 1’s credentials. This bidirectional masquerading ensures the TCP five-tuple matches the client’s socket expectation, rendering the redirection transparent.

C. Difficulty and Solutions

A significant implementation challenge was that the switch initially failed to install redirection rules, defaulting to the “send to controller” action. This was caused by a dependency on strict MAC learning where redirect rules were attempted before the destination MAC was fully resolved. To address this, the packet-processing logic was refined to prioritize basic forwarding for ARP/ICMP to complete MAC learning first. Specific redirect rules for TCP flows are only installed once the IP/MAC mapping is validated, ensuring valid flow modifications.

D. Programming Techniques

- **Modular Design:** The topology and controller logic are placed in separate Python scripts, and key steps (initialisation, host configuration, flow setup) are split into smaller blocks to reduce coupling and improve readability, namely (`networkTopo.py`, `ryu_forward.py`, `ryu_redirect.py`).
- **Encapsulation:** Repetitive tasks, such as building match fields and flow modification messages, are encapsulated in helper functions.
- **Object-Oriented Programming (OOP):** The controllers are implemented as subclasses of the Ryu base application, effectively utilizing built-in event handlers for Packet-In events.

V. TESTING AND RESULTS

A. Test Environment & Topology Initialization

The experiments were conducted using the provided `CAN201-Default.ova` image in VirtualBox. We initialized the network using `sudo python3 networkTopo.py`. **Verification:** The topology structure (Fig. 6) and host IP/MAC addresses were verified via `ifconfig` on Client, Server1, and Server2, confirming compliance with the specifications.

B. Forwarding Case (Task 4)

Controller & Connectivity: We executed the forwarding controller (`ryu_forward.py`). Connectivity was confirmed via `ping` from Client to Server1 (10.0.1.2) and Server2 (10.0.1.3). The flow tables were verified using `ovs-ofctl` to ensure entries expired after the 5-second idle timeout (Fig. 10). It is worth noting that to ensure the ARP forwarding rules could be queried, we repeated the previous steps to reinitialize the entire network and install the SDN controller. This ensured no prior requests caused the ARP rules to be cached by the virtual host. In other words, the flow table we queried was

```

"Node: client"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 client.py
TCP client sending to server...
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.2)

"Node: server_1"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 server.py
TCP server bind on 9999...
accepted ('10.0.1.5', 52544)
from client (10.0.1.5.52544): seq=0 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=1 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=2 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=3 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=4 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=5 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=6 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=7 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=8 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=9 Hello, server (10.0.1.2)
from client (10.0.1.5.52544): seq=10 Hello, server (10.0.1.2)

"Node: server_2"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 server.py
TCP server bind on 9999...

```

Fig. 3: TCP traffic correctly forwarded to Server1.

```

"Node: client"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 client.py
TCP client sending to server...
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)
From server (10.0.1.2.9999): Hello, client (10.0.1.5)! This is server (10.0.1.3)

"Node: server_1"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 server.py
TCP server bind on 9999...

"Node: server_2"
root@can201-VirtualBox: /home/can201/CH201_OI2# sudo python3 server.py
TCP server bind on 9999...
accepted ('10.0.1.5', 52562)
from client (10.0.1.5.52562): seq=0 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=1 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=2 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=3 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=4 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=5 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=6 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=7 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=8 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=9 Hello, server (10.0.1.2)
from client (10.0.1.5.52562): seq=10 Hello, server (10.0.1.2)

```

Fig. 4: TCP traffic redirected to Server2.

triggered by the very first request in this network—namely, this ping request.

TCP Traffic & Capture: The socket programs were executed (Server1/2 first, Client after 5s delay). As shown in Fig. 3, TCP traffic was correctly forwarded to Server1. The specific flow entry was verified in Fig. 12. Simultaneously, we use `tcpdump` on `client-eth0` captured the first few packages (Fig. 13) and save to a pcap file for TCP handshake latency analysis.

C. Redirection Case (Task 5)

Controller & Traffic Redirection: We replaced the controller with `ryu_redirect.py`. Following the same ping verification steps, the TCP traffic test was repeated. Despite the Client targeting Server1's IP, the traffic was successfully redirected to Server2 (Fig. 4). The modified flow entries confirming this redirection are shown in Fig. 14.

D. Performance Analysis & Conclusion

We calculated the TCP 3-way handshake latency based on the PCAP files. To ensure reliability, we conducted 10 trials for both scenarios.

The comparative results are presented in Fig. 5.

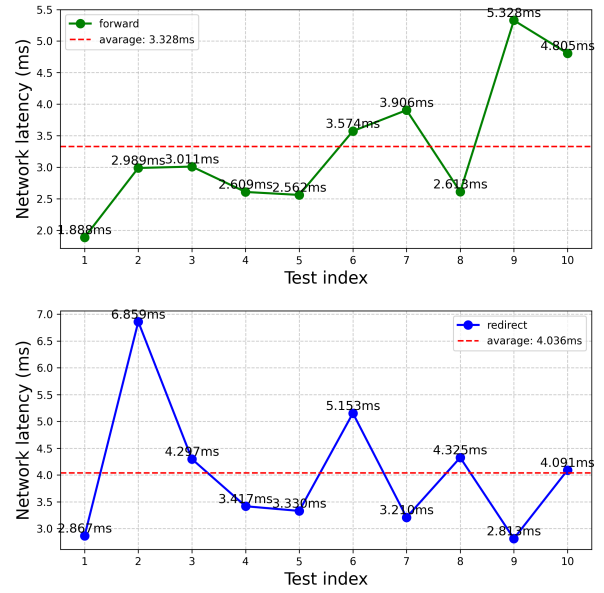


Fig. 5: Latency comparison: Forwarding vs. Redirection.

- **Forwarding Case:** Average latency of 3.328 ms.
- **Redirection Case:** Average latency of 4.036 ms.

Conclusion: The redirection scenario exhibits a slight latency increase (≈ 0.7 ms) compared to forwarding. This overhead is attributed to the additional processing time required for the controller to modify packet headers (IP/MAC rewriting) during the redirection process.

VI. CONCLUSION

In summary, this project successfully realizes SDN-based traffic management, meeting all core requirements through Mininet and Ryu. The Mininet topology is accurately configured with specified IP/MAC addresses, ensuring seamless connectivity between Client, Server1, and Server2. The two Ryu controllers work reliably, with proper flow entry management and 5-second idle timeouts. Comparative latency tests show minimal overhead from redirection, confirming SDN's feasibility for real-world use.

While the project achieves its goals, future improvements could include dynamic topology adjustments and more complex traffic policies. Overall, this work validates SDN's flexibility for traffic control and provides a practical foundation for applications like cloud load balancing and secure traffic steering.

VII. ACKNOWLEDGMENT

Yaokai Yang (2359830) contributes 20% to the project, Xiang Lin (2362086) contributes 20% to the project, Xinyang Zhang (2360067) contributes 20% to the project, Yunxuan Wu (2361506) contributes 20% to the project and Haoran Zheng (2359982) contributes 20% to the project.

REFERENCES

- [1] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [2] N. McKeown *et al.*, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [3] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 2010, pp. 1–6.
- [4] Ryu Project Team, "Ryu SDN Framework," <https://ryu-sdn.org/>, 2024. [Online]. Available: <https://ryu-sdn.org/>. [Accessed: Nov. 2025].
- [5] N. Handigol, S. Seetharaman, N. McKeown, and R. Johari, "Plug-n-Serve: Load-balancing web traffic using OpenFlow," *ACM SIGCOMM Demo*, vol. 2009, 2009.
- [6] R. Wang, D. Butnariu, and J. Rexford, "OpenFlow-based server load balancing gone wild," in *Proceedings of the 11th USENIX conference on Hot topics in management of internet, cloud, and enterprise networks and services*, 2011, pp. 12–12.

APPENDIX

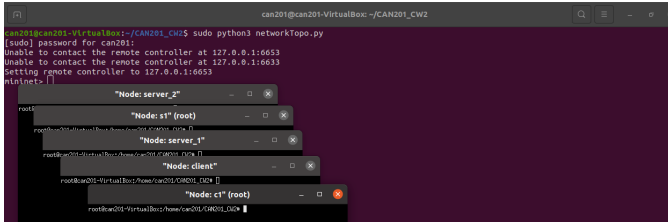


Fig. 6: Screenshot for `sudo python3 networkTopo.py`

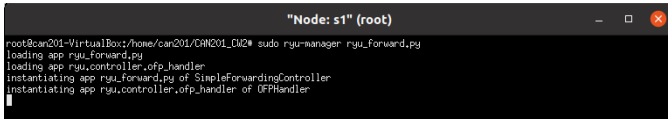


Fig. 7: Screenshot for `sudo ryu-manager ryu_forward.py`

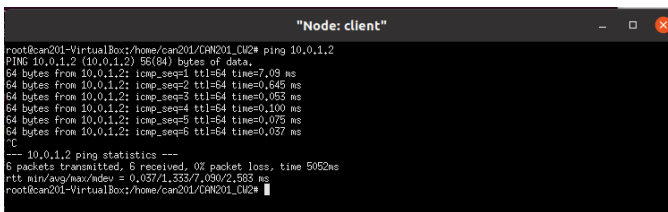


Fig. 8: Screenshot for ping 10.0.1.2 (server₁)

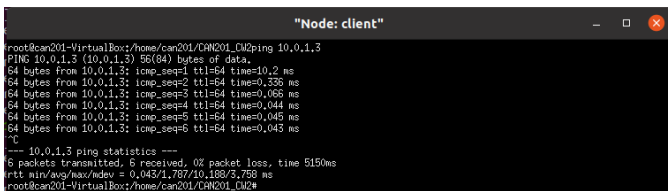


Fig. 9: Screenshot for ping 10.0.1.3 (server₂)

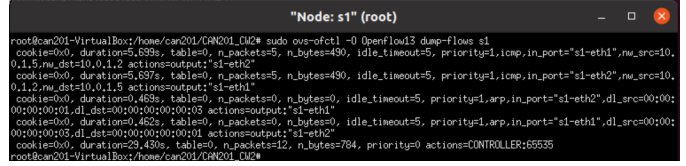


Fig. 10: The configured flow table entries (ping: client → server₁).

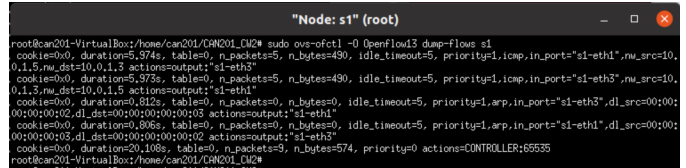


Fig. 11: The configured flow table entries (ping: client → server₂).

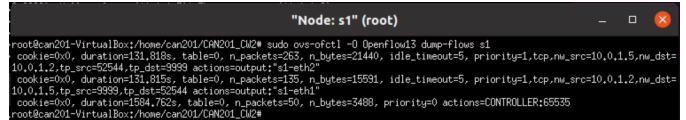


Fig. 12: The configured flow table entries (TCP forward)

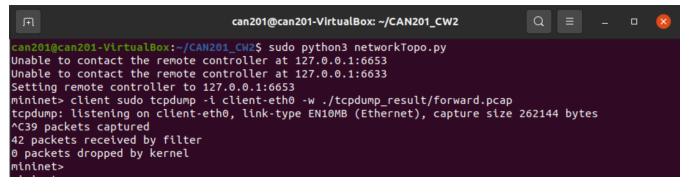


Fig. 13: The utilization of tcpdump for packet capturing

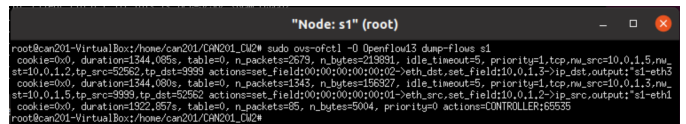


Fig. 14: The configured flow table entries (TCP redirect)